

Introduction to Data Structures

Data Structure Course
DGIST

November 1, 2025

Contents

1. What Are Data Structures?
2. Time vs Space Trade-offs
3. Asymptotic Analysis & Big-O
4. Big-O of Common Operations
5. When to Choose a Structure
6. Abstraction and ADTs
7. Summary

What Are Data Structures?

Definition

Definition

A **data structure** is a specialized format for organizing, storing, and managing data in a computer so that it can be accessed and modified efficiently.

Why We Need Them:

1. Efficiency

Right structure = milliseconds

Wrong structure = hours

2. Organization

Systematic data management

3. Reusability

Solve similar problems

4. Abstraction

Hide implementation details

Real-World Analogies

- **Bookshelf** → Array
Organized for easy retrieval
- **Filing cabinet** → Hash table
Labeled drawers
- **Stack of plates** → Stack
Take from top
- **Store line** → Queue
First come, first served

Impact on Performance

Example: Checking if Element Exists

Using List (Array):

```
1 my_list = [1, 2, ..., 1000000]
2
3 if 999999 in my_list: # O(n)
4     print("Found!")
5
6 # Must scan through all elements
7 # For 1M elements: slow!
```

Using Set (Hash Table):

```
1 my_set = {1, 2, ..., 1000000}
2
3 if 999999 in my_set: # O(1)
4     print("Found!")
5
6 # Direct lookup
7 # For 1M elements: instant!
```

Key Insight

For 1 million elements, set lookup is nearly instantaneous while list scan takes significant time. **Choosing the right data structure matters!**

Time vs Space Trade-offs

The Fundamental Trade-off

Core Principle

Often, we can make an algorithm **faster** by using **more memory**, or **save memory** by accepting **slower execution**.

Time Complexity

How runtime grows with input size

- Focus: Speed
- Measure: Operations
- Goal: Minimize time

Space Complexity

How memory usage grows with input size

- Focus: Memory
- Measure: Bytes/storage
- Goal: Minimize space

Reality Check

There's **rarely** a solution that is both the fastest AND uses the least memory. Engineers must choose based on constraints.

Trade-off Example 1: Memoization

Without Memoization

Slower, Less Memory

```
1 def fibonacci(n):
2     if n <= 1:
3         return n
4     return fibonacci(n-1) + fibonacci(n-2)
5
6 # Time:  $O(2^n)$  - exponential!
7 # Space:  $O(n)$  - recursion stack
8 # fib(40) takes several seconds
```

With Memoization

Faster, More Memory

```
1 cache = {}
2 def fibonacci_memo(n):
3     if n in cache:
4         return cache[n]
5     if n <= 1:
6         return n
7     result = fibonacci_memo(n-1) + fibonacci_memo(n-2)
8     cache[n] = result
9     return cache[n]
10
11 # Time:  $O(n)$  - much better!
12 # Space:  $O(n)$  - cache storage
13 # fib(40) is instant
```

Trade-off: Used extra memory (cache) to gain massive speed improvement

Trade-off Example 2: In-place vs Out-of-place

Out-of-place Algorithm

More Space, Simpler

```
1 def reverse_list(arr):
2     return arr[::-1]
3
4 # Creates new array
5 # Space: O(n)
6 # Original preserved
```

Original



New Array



In-place Algorithm

Less Space, More Complex

```
1 def reverse_inplace(arr):
2     left, right = 0, len(arr)-1
3     while left < right:
4         temp = arr[left]
5         arr[left] = arr[right]
6         arr[right] = temp
7         left += 1
8         right -= 1
9
10 # Modifies original
11 # Space: O(1)
```

When to Choose What?

Optimize for Time When:

- Dealing with large datasets
- User experience matters
- Memory is abundant
- Real-time requirements
- Interactive applications

Optimize for Space When:

- Memory is limited
- Embedded systems
- Mobile devices
- Data too large for RAM
- Resource-constrained devices

Examples

- Web servers (caching)
- Video games
- Trading systems
- Search engines

Examples

- IoT devices
- Microcontrollers
- Big data processing
- Satellite systems

Asymptotic Analysis & Big-O

Why Asymptotic Analysis?

The Problem with Exact Counts

Actual runtime depends on:

- Hardware (CPU speed, memory)
- Programming language
- Compiler optimizations
- Current system load

The Solution: Asymptotic Analysis

Focus on **growth rate** as input size approaches infinity

- Hardware-independent
- Language-independent
- Describes **scalability**
- Constants become insignificant with large n

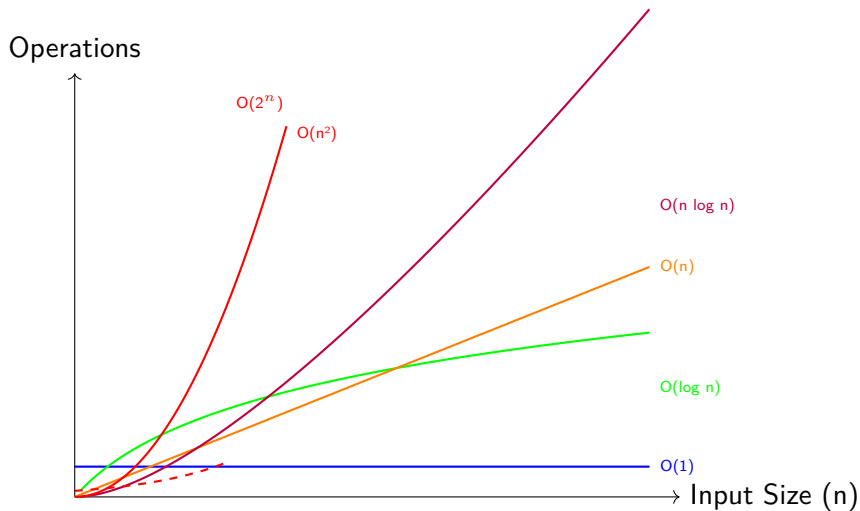
Common Big-O Classes

Notation	Name	Example	n=100
$O(1)$	Constant	Array access, hash lookup	1
$O(\log n)$	Logarithmic	Binary search	≈ 7
$O(n)$	Linear	Linear search, scan	100
$O(n \log n)$	Linearithmic	Merge sort, quicksort	≈ 664
$O(n^2)$	Quadratic	Nested loops, bubble sort	10,000
$O(n^3)$	Cubic	Triple nested loops	1,000,000
$O(2^n)$	Exponential	Recursive fibonacci	$\approx 10^{30}$
$O(n!)$	Factorial	All permutations	$\approx 10^{157}$

Visual Scale

For $n = 100$, $O(2^n)$ is **universe-ending!** Exponential growth is catastrophic.

Big-O Growth Visualization



From fastest (top) to slowest (bottom)

Big-O Rules

Rule 1: Drop Constants

$$O(2n) \rightarrow O(n)$$

$$O(100) \rightarrow O(1)$$

$$O(n/2) \rightarrow O(n)$$

Why? Constants don't affect growth rate

Rule 2: Drop Lower-Order Terms

$$O(n^2 + n) \rightarrow O(n^2)$$

$$O(n^2 + 100*n + 50) \rightarrow O(n^2)$$

$$O(n^3 + n^2 + n) \rightarrow O(n^3)$$

Why? Highest order dominates for large n

Rule 3: Different Variables

Two inputs? Use different variables!

Sequential: $O(a + b)$

Nested: $O(a * b)$

Rule 4: Worst Case

Big-O Examples

$O(1)$ - Constant

```
1 def get_first(arr):
2     return arr[0]
3 # Always 1 operation
```

$O(n)$ - Linear

```
1 def find_max(arr):
2     max_val = arr[0]
3     for num in arr: # n iterations
4         if num > max_val:
5             max_val = num
6     return max_val
```

$O(n^2)$ - Quadratic

```
1 def bubble_sort(arr):
2     n = len(arr)
3     for i in range(n): # n times
```

$O(\log n)$ - Logarithmic

```
1 def binary_search(arr, target):
2     left, right = 0, len(arr) - 1
3     while left <= right:
4         mid = (left + right) // 2
5         if arr[mid] == target:
6             return mid
7         elif arr[mid] < target:
8             left = mid + 1
9         else:
10            right = mid - 1
11            # Halves search space each time!
12    return -1
```

$O(n \log n)$ - Linearithmic

```
1 def merge_sort(arr):
2     if len(arr) <= 1:
3         return arr
4     mid = len(arr) // 2
5     left = merge_sort(arr[:mid])
```


Big-O of Common Operations

Sorting Algorithms Comparison

Algorithm	Average	Worst	Space	Stable?
Quick Sort	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Bubble Sort	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Insertion Sort	$O(n^2)$	$O(n^2)$	$O(1)$	Yes

Quick Selection Guide

- Need **fast random access**? → Arrays
- Need **fast insertion/deletion at beginning**? → Linked lists
- Need **fast key-value lookup**? → Hash tables
- Need **sorted data with fast operations**? → Balanced BSTs
- Need **LIFO access**? → Stacks

When to Choose a Structure

Decision Framework

Questions to Ask

1. What operations will be most frequent?

- Lookups? → Hash table or BST
- Insertions/deletions? → Linked list or balanced tree
- Processing in order? → Queue
- Need to undo? → Stack

2. Do you need ordered data?

- Yes, sorted → BST, sorted array
- Yes, insertion order → Array, linked list
- No → Hash table (fastest)

3. Do you know the size in advance?

- Yes, fixed → Array
- No, dynamic → Linked list, dynamic array, hash table

4. What are your constraints?

- Limited memory → Space-efficient structures

- Need speed → Time-efficient structures

Real-World Scenario 1: Phone Book

Requirements

- Fast lookup by name
- Alphabetical order useful
- Operations:
 - Search (frequent)
 - Insert (rare)
 - Delete (rare)

Decision

Hash table for $O(1)$ lookup
or **BST** for $O(\log n)$ with ordering

Implementation

```
1 # Using hash table
2 phone_book = {
3     "Alice": "555-1234",
4     "Bob": "555-5678"
5 }
6
7 # O(1) lookup
8 number = phone_book["Alice"]
```

Real-World Scenario 2: Browser History

Requirements

- Recent pages accessed frequently
- Navigate back and forward
- Operations:
 - Push new page
 - Go back (pop)
 - Go forward (pop)

Decision

Two stacks:

- Back stack
- Forward stack

Implementation

```
1 back_stack = []
2 forward_stack = []
3
4 def visit_page(url):
5     back_stack.append(current_page)
6     forward_stack.clear()
7     current_page = url
8
9 def go_back():
10     if back_stack:
11         forward_stack.append(current_page)
12         current_page = back_stack.pop()
13
14 def go_forward():
15     if forward_stack:
16         back_stack.append(current_page)
17         current_page = forward_stack.pop()
```

Real-World Scenario 3: Print Queue

Requirements

- First document submitted prints first
- Operations:
 - Add to queue (frequent)
 - Remove from queue (frequent)
- FIFO behavior essential

Decision

Queue

(First In, First Out)

Implementation

```
1 from collections import deque
2
3 print_queue = deque()
4
5 # Enqueue documents
6 print_queue.append("doc1.pdf")
7 print_queue.append("doc2.pdf")
8 print_queue.append("doc3.pdf")
9
10 # Dequeue - FIFO
11 current_job = print_queue.
    popleft()
12 # "doc1.pdf" prints first
```

Common Pitfalls

What NOT to Do

- ✗ Using lists for frequent lookups
($O(n)$ instead of $O(1)$ with hash table)
- ✗ Using arrays when size is unknown
(Costly resizing)
- ✗ Using BST when order doesn't matter
(Hash table is faster)
- ✗ Using recursive structures without considering stack overflow
(Deep recursion can crash)

General Guidelines

- **Default to hash tables** for key-value storage (most versatile)
- **Use arrays** when you need index-based access

Abstraction and ADTs

Abstract Data Types (ADT)

Definition

An **ADT** defines a data type by its **behavior** (operations) rather than its implementation. It specifies **what** operations can be performed, not **how**.

Key Principle

Separate the **interface** (what operations are available) from the **implementation** (how they work internally)

Benefits

1. Flexibility

Change implementation without affecting users

2. Simplicity

Hide internal complexity

Real-World Analogy

ADT = Car Interface

- Steering wheel
- Gas pedal

ADT Example: Stack

Stack ADT Interface

```
1 class StackADT:
2     def push(self, item): pass
3     def pop(self): pass
4     def peek(self): pass
5     def is_empty(self): pass
6
7 # Behavior: LIFO
8 # (Last In, First Out)
```

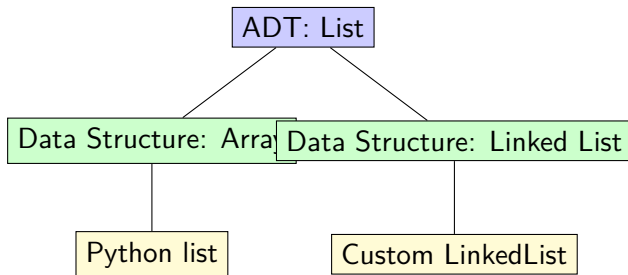
Implementation 1: Array

```
1 class ArrayStack(StackADT):
2     def __init__(self):
3         self._data = []
4
5     def push(self, item):
6         self._data.append(item)
7
8     def pop(self):
9         return self._data.pop()
```

Implementation 2: Linked List

```
1 class LinkedStack(StackADT):
2     def __init__(self):
3         self._head = None
4         self._size = 0
5
6     def push(self, item):
7         new_node = Node(item, self._head)
8         self._head = new_node
9         self._size += 1
10
11    def pop(self):
12        if not self._head:
13            raise IndexError("empty")
14        value = self._head.value
15        self._head = self._head.next
16        self._size -= 1
17        return value
18
19    def peek(self):
20        if not self._head:
21            raise IndexError("empty")
22        return self._head.value
23
24    def is_empty(self):
```

ADT Hierarchy



Three Levels

- **ADT** (Abstract): The concept/interface (e.g., "List")
- **Data Structure** (Logical): How data is organized (e.g., "Array" or "Linked List")
- **Implementation** (Physical): Actual code in a programming language

Encapsulation in ADTs

Good: Hidden Implementation

```
1 class Stack:
2     def __init__(self):
3         self._data = [] # Private
4
5     def push(self, item):
6         self._data.append(item)
7
8     def pop(self):
9         return self._data.pop()
10
11 # Users can't access _data directly
12 # Implementation can change!
```

- ✓ Implementation hidden
- ✓ Can swap to linked list easily
- ✓ Users can't break invariants

Bad: Exposed Implementation

```
1 class BadStack:
2     def __init__(self):
3         self.data = [] # Public!
4
5     def push(self, item):
6         self.data.append(item)
7
8     def pop(self):
9         return self.data.pop()
10
11 # Problem:
12 stack = BadStack()
13 stack.data = "oops!" # Breaks it!
14 stack.data.clear()   # Ruins stack!
```

- ✗ Implementation exposed
- ✗ Users can break the stack
- ✗ Hard to change implementation

Summary

Key Takeaways

1. Data Structures Matter

- Right structure = milliseconds, Wrong structure = hours
- Choose based on most frequent operations
- Understand time vs space trade-offs

2. Big-O Analysis

- Describes growth rate, not exact time
- Focus on scalability for large inputs
- Remember: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$
- Drop constants and lower-order terms

3. Common Operations

- Arrays: $O(1)$ access, $O(n)$ insert/delete in middle
- Hash tables: $O(1)$ average lookup/insert/delete

Key Takeaways (continued)

4. Choosing Structures

- Ask: What operations are most frequent?
- Ask: Do you need ordering?
- Ask: What are your constraints (time/space)?
- Default to hash tables for key-value storage
- Use specialized structures when they match your pattern

5. Abstract Data Types

- Separate interface from implementation
- Hide internal details (encapsulation)
- Makes code flexible and maintainable
- Same interface, multiple implementations

Thank You!

Questions?

*“Choosing the right data structure
is half the battle in algorithm design.”*